vulpes



PLANO · RASCUNHO PARA DECISÃO

Vulpes Lusitanus 1.0

Um modelo feito de raiz que lê, escreve e explica código, em inglês e em português, e que é o cérebro de um assistente que corre no teu PC.

14 de novembro de 2026 · revisto (até 14/11 chamava-se Aupus Vulpes, e a primeira versão, de 13/11, Aupus 1)

Vem depois do Vulpes Flos 1.0 (348M, treinado no Kaggle; plano à parte) e do Vulpes 0.1 (125M, treinado a 10/11/2026)

Todos os custos e resultados futuros são estimativas, a confirmar pelo piloto.

EM RESUMO

Do Vulpes 0.1 a um assistente de programação teu

O Vulpes Lusitanus 1.0 é o modelo grande da família, feito como o 0.1: tokenizer, dados, treino, fine-tuning e inferência escritos neste repositório, em PyTorch simples, sem pesos de outros modelos. Vem depois do Vulpes Flos 1.0, que melhora o 0.1 sem custos. O objetivo muda: deixa de ser só "escrever inglês" e passa a ser **perceber código, seguir instruções em português e usar ferramentas**.

1,07 B	8 192	50–100 B	1–2,9 mil
parâmetros	**tokens de contexto**	**tokens de treino**	**USD, estimativa**
8,5× o 0.1 e 3× o Flos 1.0; cabe numa GPU de 4 GB para conversar	8× mais; cerca de 600 linhas de Python em vez de ~40	20–40× o 0.1; inglês, código e português	GPUs alugadas (D2); 0 se o TPU Research Cloud o aceitar (D8)

Três peças

O cérebro

Lusitanus Base e **Lusitanus Instruct**: um modelo de 1,07B com tokenizer novo, contexto de 8k e treino em três línguas (inglês, português, código).

O código

Dados de código com licenças permissivas, preenchimento a meio (FIM) para autocompletar e, no fim, **aprendizagem por reforço com testes**: o modelo só é recompensado por código que funciona.

O corpo

O **Agente Vulpes**: o programa à volta do modelo que lê ficheiros, procura, propõe alterações e corre testes, com a tua confirmação, a partir do chat que já existe.

Como se chega lá

Em quatro fases e doze marcos (página 11), que começam quando o Vulpes Flos 1.0 acabar. A regra mais importante: **nada de caro sem um piloto barato primeiro**. Um modelo de 321M (o "Lusitanus Mini", ~50–100 USD) passa por todas as etapas antes de se alugar o treino grande, tal como o piloto de uma hora fez antes do 0.1. O Flos 1.0 deixa prontos o treino em várias GPUs (DDP), a agenda WSD e o fp16.

> **Expectativa honesta.** Um modelo de 1B treinado com um orçamento pessoal não chega aos assistentes comerciais. O que se pode esperar: autocompletar e escrever funções pequenas, explicar código curto, responder em português, e usar ferramentas simples (ler, procurar, correr testes) num ciclo curto. Vai inventar coisas, sobretudo em tarefas longas, e por isso o agente confirma tudo o que muda ficheiros.

O que fica decidido aqui e o que é teu

Este plano propõe a forma do modelo, os dados, o treino e o agente. Há oito decisões que são tuas (página 13), entre elas o orçamento e onde treinar, a regra "sem dados gerados por LLMs" aplicada ao código, e português europeu ou do Brasil. As GPUs gratuitas do Kaggle não chegam para o Lusitanus (página 7): servem para o Flos.

1 · PONTO DE PARTIDA

O que o Vulpes 0.1 nos ensinou

O 0.1 provou que a cadeia inteira funciona, do tokenizer ao chat no browser. Também mostra, com números, porque não chega para programar. O Vulpes Flos 1.0 vai torná-lo maior, mas continua a ter os mesmos quatro limites: corrigi-los é o trabalho do Lusitanus.

Resultados medidos

	VULPES 0.1 BASE	VULPES 0.1 INSTRUCT	AO ACASO
HellaSwag (acc_norm, 10 042 perguntas)	31,3%	31,2%	25%
ARC-Easy (acc, 2 376)	52,4%	52,2%	25%
PIQA (acc_norm, 1 838)	61,9%	61,0%	50%
Perplexidade de validação (FineWeb-Edu)	22,9	24,8	32 000

Fonte: `runs/vulpes-0.1-*/eval_best.json`. Base: 2,5B tokens, 9 537 passos, ~4 h numa A100 80 GB a ~167 mil tokens/s (138 TFLOPS efetivos). Instruct: 600 passos de SFT.

Os quatro limites que importam para programar

1. Contexto curto

1 024 tokens. Com o tokenizer atual, isso é cerca de 40 linhas de Python. Não dá para ler um ficheiro, quanto mais um projeto.

2. Nunca viu código

O pré-treino foi só FineWeb-Edu: páginas educativas em inglês. Sabe que código existe, mas não o sabe escrever.

3. O tokenizer é inglês

Medido numa frase de exemplo de cada: inglês **4,5** caracteres por token, português **2,8**, Python **1,85**. Código e português gastam 1,6–2,4× mais contexto e mais treino pela mesma informação.

4. Não sabe agir

O formato de chat só tem `system`, `user` e `assistant`. Não há maneira de pedir para ler um ficheiro nem de receber o resultado.

O que se mantém

- **A arquitetura** (RMSNorm, RoPE, GQA, SwiGLU, embeddings partilhados): só muda de tamanho. `vulpes/model/` serve quase sem alterações.
- **O treino**: AdamW, retoma bit-a-bit, Ctrl-C seguro, `--stop-after` para o orçamento, `scripts/runpod/` com cópia verificada para o PC; e, do Vulpes Flos 1.0, DDP, WSD e fp16.
- **As regras**: dados escritos por pessoas e com licença aberta, nada de `transformers/peft/trl`, código feito para ser lido.
- **A interface**: o chat no browser e no terminal, que passam a ser a cara do agente.

2 · ARQUITETURA

O mesmo desenho, maior

Fica a família Llama que o 0.1 e o Vulpes Flos 1.0 usam. Muda o tamanho, o vocabulário e o contexto. Antes do modelo grande há um modelo piloto, o Lusitanus Mini, que testa tudo a um sexto do custo.

	VULPES 0.1	VULPES FLOS 1.0	LUSITANUS MINI (PILOTO)	VULPES LUSITANUS 1.0
Parâmetros (sem embeddings)	125,3M (100,7M)	348,4M (315,7M)	320,9M (270,6M)	1 069,6M (969,0M)
Camadas × largura	16 × 768	28 × 1 024	24 × 1 024	22 × 2 048
Cabeças de query / de KV (GQA)	12 / 4	16 / 4	16 / 4	32 / 4
Dimensão por cabeça	64	64	64	64
SwiGLU (camada escondida)	2 048	2 816	2 816	5 632
Vocabulário	32 000	32 000	49 152	49 152
Contexto	1 024	2 048	4 096	4 096 → 8 192
RoPE θ	10 000	10 000	500 000	500 000
FLOP por token de treino	0,83 G	2,44 G	2,53 G	7,53 G (8,63 G a 8k)
Tokens de pré-treino	2,5 B	~10 B	10 B	50–100 B

Parâmetros e FLOP calculados com `vulpes.model.config.param_count` e `flops_per_token`, a mesma contabilidade do 0.1.

Porquê esta forma

1B, não 350M nem 3B

Abaixo de ~1B, os modelos publicados quase não escrevem código que funcione: por isso o Vulpes Flos 1.0 (348M) fica pelo inglês. Acima, o custo sai de um orçamento pessoal: um 3B com o mesmo rácio de dados custa ~8× mais. 1B é o menor tamanho em que o objetivo é realista.

22 × 2 048, SwiGLU 5 632

É a forma do TinyLlama 1.1B, um modelo aberto conhecido por treinar estável a este tamanho. Copiar uma forma provada tira uma variável de um treino caro.

Só 4 cabeças de KV

A cache de atenção é o que limita o contexto numa GPU pequena. Com 4 cabeças de KV, 8 192 tokens ocupam ~185 MB em fp16, o que é decisivo para o modelo correr no teu GTX 1650 Super (página 9).

Vocabulário de 49 152

Mais espaço para português e código sem sair de `uint16` (máximo 65 536), por isso os shards continuam a ocupar 2 bytes por token e o carregador de dados não muda.

θ = 500 000 desde o início

Um θ maior faz o RoPE rodar mais devagar, o que facilita treinar a 4k e estender para 8k no fim sem truques (como fez o Llama 3).

Máscara por documento

A 4k–8k tokens, uma janela junta vários ficheiros. Opcional: impedir a atenção de atravessar fronteiras de documento (o 0.1 não o fazia, decisão 19). Decide-se no piloto, medindo a perda.

3 · TOKENIZER V2

Um tokenizer que fala três línguas

O BPE do 0.1 continua: mesmo código, mesmo formato, mesma pré-tokenização sem dependências. Muda o que ele lê para aprender e o tamanho do vocabulário.

Treino

- **Amostra de ~1 GB** misturada: 40% inglês (FineWeb-Edu), 35% código (sobretudo Python, depois JavaScript/TypeScript, shell, SQL, HTML/CSS) e 25% português (FineWeb-2).
- **49 152 ids**: 256 bytes, ~48 832 fusões e **64 lugares especiais**. Hoje usam-se 14; os outros 50 ficam reservados, para que acrescentar um token numa versão seguinte não obrigue a refazer os dados.
- **Indentação**: a regra `\s+(?!\S)` já separa blocos de espaços; com código na amostra, o BPE aprende tokens para 4, 8 e 12 espaços sozinho. A testar com a medição abaixo antes de mexer na regex.
- **Ficheiros sempre em LF** e SHA-256 registado (decisão 37 do 0.1). O tokenizer novo tem outro SHA, por isso os dados e checkpoints do 0.1 e do Vulpes Flos 1.0 (que mantém o tokenizer de 32k) ficam separados de propósito.

Metas, medidas em texto que não entrou no treino

	HOJE (0.1)	META (V2)	PORQUÊ
Inglês	4,56	≥ 4,3	Pode perder um pouco: o vocabulário passa a ser partilhado
Português	2,8*	≥ 3,8	Respostas em PT mais rápidas e com mais contexto
Python	1,85*	≥ 3,0	Um ficheiro de 600 linhas tem de caber em 8k tokens

Caracteres por token. * Medido numa frase de exemplo a 13/11/2026; o valor de inglês é o do conjunto de teste do tokenizer (`vulpes-bpe-32k.stats.json`).

Tokens especiais

TOKEN	PARA QUÊ
`<|bos|> <|eos|> <|pad|>`	Como no 0.1
`<|system|> <|user|> <|assistant|> <|end|>`	Formato de chat, como no 0.1
`<|tool_call|> <|tool_result|>`	O modelo pede uma ferramenta; o agente devolve o resultado
`<|fim_prefix|> <|fim_suffix|> <|fim_middle|>`	Preencher código a meio de um ficheiro (autocompletar no editor)
`<|file|>`	Separa ficheiros do mesmo repositório numa janela de treino
`<|reserved_0|> … <|reserved_49|>`	Para versões futuras

Como hoje, `encode` trata estes textos como texto normal, salvo com `allow_special=True`. Isto passa a ser ainda mais importante: um ficheiro lido pelo agente não pode fingir ser um resultado de ferramenta.

4 · DADOS

100 mil milhões de tokens, escritos por pessoas

Três fontes, todas com licenças abertas e todas de texto ou código escrito por pessoas. As proporções valem para qualquer orçamento: com 50B tokens, cada fonte dá metade.



55 B Inglês · 55% **30 B** Código · 30% **15 B** Português · 15%

Inglês · FineWeb-Edu Código · StarCoderData Português · FineWeb-2

Mistura de pré-treino para 100B tokens. Os valores exatos ajustam-se no piloto, olhando para a perda de validação de cada fonte.

FONTE	O QUE É	LICENÇA	TOKENS
FineWeb-Edu subconjunto sample-100BT	Páginas educativas em inglês, como no 0.1, mas uma amostra 10× maior. Nenhum token repete.	ODC-By 1.0	55 B
StarCoderData do The Stack, só licenças permissivas	Código do GitHub (Python ~40% do que usamos, D6), notebooks Jupyter, issues e commits. É preciso aceitar os termos no Hugging Face e respeitar os pedidos de remoção dos autores.	Permissivas (MIT, Apache, BSD…)	30 B
FineWeb-2 por_Latn	Páginas web em português. Maioritariamente do Brasil (decisão D4).	ODC-By 1.0	15 B

O que a pipeline ganha

Filtros de código

Fora ficheiros gerados automaticamente, minificados, com linhas enormes, quase só dados (JSON, CSV gigantes) ou sem licença permissiva. Os filtros Gopher do 0.1 continuam para o texto.

Segredos e dados pessoais

Código público tem chaves de API, palavras-passe e e-mails. Um filtro próprio troca-os por marcadores (`<KEY>`, `<EMAIL>`) antes de tokenizar, para o modelo nunca os aprender.

Descontaminação

HumanEval e MBPP estão copiados em muitos repositórios. Qualquer documento que partilhe um bloco de 13 palavras com um teste de avaliação sai, ou os números do capítulo 8 ficam inflacionados.

Deduplicação

As três fontes já vêm deduplicadas. Fica o hash exato do 0.1 em tudo; o MinHash (que precisaria de ~50 GB de RAM a esta escala) só se o piloto encontrar duplicados.

FIM

Metade dos ficheiros de código é reordenada como prefixo-sufixo-meio, como no StarCoder. O modelo aprende a completar no meio de um ficheiro sem perder a escrita normal.

Mistura retomável

O carregador escolhe a fonte de cada janela com pesos fixos e uma semente, e guarda a posição em cada fonte. A retoma continua bit-a-bit, como hoje.

Escala: ~200 GB de shards para 100B tokens. Como no 0.1 (decisão 48), os dados constroem-se num pod com muitos CPUs: o tokenizer em Python puro faz ~0,83 M tokens/s por processo, ou seja, algumas horas com 32 processos.

5 · PRÉ-TREINO

Treino em três etapas

Primeiro o piloto barato, depois o modelo a 4k, por fim uma etapa curta a 8k.

ETAPA	MODELO	TOKENS	CONTEXTO	PASSOS	PARA QUÊ
P1 · Piloto	Mini, 321M	10 B	4 096	~19 000	Prova tokenizer, mistura, FIM, DDP, custo real por token
P2 · Base	Lusitanus, 1,07B	50–100 B	4 096	48–95 mil	O treino principal
P3 · Contexto longo	Lusitanus	~2 B	8 192	~1 900	Ficheiros inteiros; mais código e documentos longos na mistura

Lote de 256 sequências × 4 096 = 1 048 576 tokens por passo (P1: 524 288).

Hiperparâmetros

AdamW como no 0.1 (β 0,9/0,95, decay 0,1 só nas matrizes, clip 1,0). **LR máxima 3×10^{-4}** (metade do 0.1: modelos maiores querem LR menor), aquecimento de 2 000 passos.

Agenda WSD (aquecimento, patamar, descida) em vez de cosseno. Com o cosseno, o fim do treino tem de ser escolhido no início. Com WSD, a descida só começa nos últimos ~10%, por isso **começa-se com 50B e decide-se a meio se se estende a 100B**, sem recomeçar. Continua a ser uma função pura do passo (decisão 21).

Custo

ETAPA	A100 80 GB	H100 80 GB	USD (H100)
P1 · Piloto, 10B	51 h	23 h	50–70
P2 · Base, 50B	757 h	348 h	700–1 050
P2 · Base, 100B	1 515 h	697 h	1 400–2 100
P3 · Contexto 8k, 2B	35 h	16 h	30–50
SFT + RL (capítulo 7)	—	~30–60 h	60–180
Total com 50B (+20% de margem)			**~1 000–1 600**
Total com 100B (+20% de margem)			**~1 850–2 900**

A100: 138 TFLOPS efetivos, **medidos** no treino do 0.1. H100: 300 TFLOPS, **calculados, não medidos**; o P1 mede o valor real. Preços de referência: A100 1,2–1,9 USD/h, H100 2–3 USD/h; confirmar antes de alugar.

> **Sem pagar?** O Kaggle não serve: 1B com AdamW ocupa ~17 GB por GPU (uma T4 tem 16) e 50B tokens seriam ~2 anos de quota. A única hipótese grátis é o TPU Research Cloud: um v4-8 faria os 50B em ~10 dias, mas obriga a portar o treino para torch_xla ou JAX (D8).

O código de treino ganha mistura, FIM e perda por fonte. DDP e WSD já vêm testados do Vulpes Flos 1.0. P2: ~2–4 semanas numa H100 ou ~2–4 dias em oito, pelo mesmo preço (D3).

6 · INSTRUCT

Ensinar a conversar, a programar e a agir

O Base continua texto. O Instruct responde, escreve código a pedido e sabe pedir uma ferramenta. Chega-se lá em duas fases: exemplos escritos por pessoas, depois prática com testes que dizem se o código funciona.

Fase 1 · Fine-tuning supervisionado (SFT)

DADOS	O QUE ENSINAM	LICENÇA
oasst2, inglês **e português**	Conversar; como no 0.1, agora também com as conversas em português	Apache-2.0
CommitPackFT	Pegar num pedido (a mensagem de um commit real) e alterar código em conformidade	Permissivas
Docstring → função	Escrever a função que uma descrição pede. Extraído do próprio StarCoderData, sem dados novos	Permissivas
Identidade (PT e EN)	Quem é o Vulpes. Escrito por ti, como `data/identity.jsonl`	Apache-2.0
~200 trajetórias de agente	Quando pedir uma ferramenta e como usar o resultado. Escritas por ti, com o agente a correr as ferramentas a sério	Apache-2.0
Dolly 15k	Instruções curtas (só se D5 o permitir, como no 0.1)	CC BY-SA 3.0

O formato, com uma ferramenta

```
<|user|>Porque é que o teste test_parse falha?<|end|>
<|assistant|><|tool_call|>{"tool": "run_tests", "args": {"path": "tests/test_parse.py"}}<|end|>
<|tool_result|>FAILED test_parse - AssertionError: assert parse("1,5") == 1.5 ...<|end|>
<|assistant|>O parse não aceita vírgula decimal. Proponho trocar a vírgula por ponto antes de converter: ...
<|end|>
```

A perda conta só nos tokens do assistente, como no 0.1. Os resultados das ferramentas são contexto, não algo a imitar.

Fase 2 · Aprendizagem por reforço com testes

Para cada problema, o modelo escreve **8 soluções**. O agente corre os testes de cada uma num ambiente isolado (sem rede, tempo limitado). As soluções que passam são reforçadas, as que falham são desencorajadas, em relação à média do grupo (o método GRPO, escrito em PyTorch simples).

Não precisa de dados de outro modelo: o professor é o interpretador de Python.

Problemas para praticar

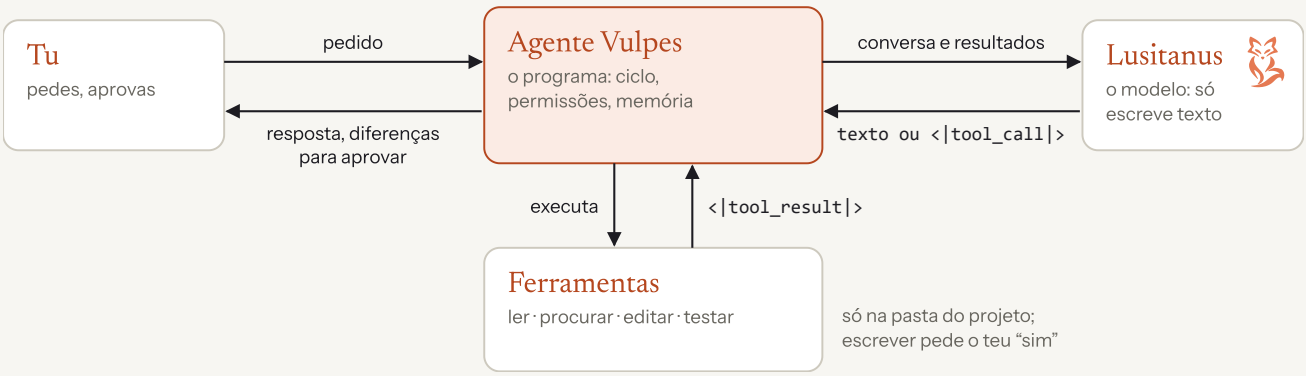
- **MBPP**, partes de treino e validação (CC BY 4.0): ~460 problemas simples, escritos por pessoas, com testes.
- **APPS**, parte de treino (MIT): ~5 000 problemas de concursos de programação, com testes.
- As partes de teste de MBPP e HumanEval nunca entram: são só para medir.

> **Ponto a decidir (D1).** No RL, o Lusitanus treina com as suas **próprias** respostas, depois de verificadas por testes. Não é texto de outro LLM, mas também não é escrito por pessoas. O plano propõe tratar isto como permitido.

7 · O CORPO

O Agente Vulpes: o teu “Jarvis”

O modelo só escreve texto. Quem lê ficheiros, corre testes e mostra alterações é um programa à volta dele. Esse programa é software normal: pode começar já, com o Vulpes 0.1 ou o Vulpes Flos 1.0, e recebe o Lusitanus quando estiver pronto.



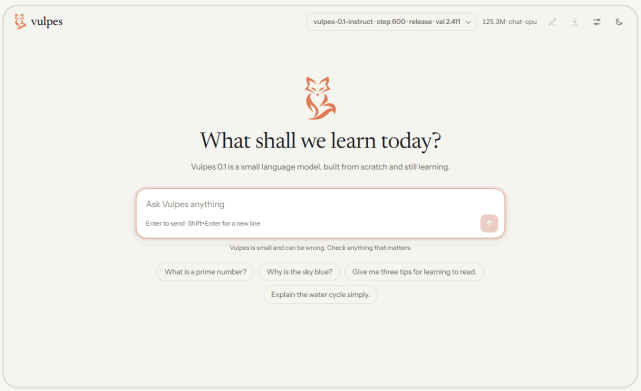
O ciclo do agente. O modelo decide; o agente executa, dentro de uma pasta que escolhes, e pede-te confirmação antes de mudar ficheiros.

Ferramentas da primeira versão

FERRAMENTA	LIMITE
`list_dir, read_file, search`	Só dentro da pasta do projeto escolhida
`write_file`	Mostra a diferença e espera pelo teu “sim”
`run_python, run_tests`	Ambiente isolado, sem rede, tempo máximo
`remember, recall`	Notas locais entre conversas, num ficheiro teu

Sem acesso à internet nem a comandos arbitrários na v1. Um modelo pequeno engana-se, e o agente parte desse princípio.

A interface já existe



O chat no browser e no terminal ganham um painel com cada chamada de ferramenta e o seu resultado, e botões para aceitar ou recusar alterações.

A correr no teu PC (GTX 1650 Super, 4 GB)

Pesos em fp16: **2,1 GB**. Cache de 8 192 tokens: **~185 MB**. Cabe, com margem para o ambiente de trabalho. Como esta GPU é lenta em fp16 (decisão 13), os pesos guardam-se em fp16 e as contas fazem-se em fp32, camada a camada; a alternativa é int8. Estimativa a medir: **15–40 tokens/s** a escrever e ~10 s para ler um contexto cheio.

8 · AVALIAÇÃO

Como se mede, e o que esperar

Os três testes do 0.1 continuam, para comparar. Juntam-se testes de código, que correm o código escrito pelo modelo, e um teste em português.

TESTE	MEDE	0.1 (MEDIDO)	LUSITANUS (ESPERADO)
HellaSwag (acc_norm)	Senso comum	31%	45–55%
ARC-Easy (acc)	Ciência escolar	52%	58–68%
PIQA (acc_norm)	Física do dia a dia	62%	69–73%
HumanEval (pass@1)	Escrever funções Python que passam testes	não medido	5–10% (Base) 8–15% (Instruct)
MBPP, parte de teste (pass@1)	Idem, problemas mais simples	não medido	10–20%
Teste em português	Compreensão em PT (a escolher, D7)	—	a definir
Tarefas do agente (nosso)	~50 tarefas reais em projetos pequenos: encontrar, explicar, corrigir	—	a medir

As expectativas vêm de modelos publicados de tamanho e treino parecidos (Pythia-1B, TinyLlama 1.1B, StarCoderBase-1B), não são promessas. O StarCoderBase-1B, treinado com 1 bilião de tokens só de código, chega a ~15% no HumanEval; o Lusitanus vê 30× menos código. O Vulpes Flos 1.0 terá os seus próprios números nestes três primeiros testes antes de o Lusitanus começar.

Um harness de execução

O código em `vulpes/eval/` mede probabilidades. Os testes de código precisam de outra peça: gerar uma solução, corrê-la isolada com tempo limite e contar quantas passam. É a mesma peça que o RL usa, por isso escreve-se uma vez.

O que vai conseguir, e o que não

Deve conseguir

- Completar a linha ou a função que estás a escrever
- Escrever funções pequenas a partir de uma descrição
- Explicar o que faz um ficheiro curto, em português
- Ler o erro de um teste e apontar a causa provável
- Procurar num projeto pequeno e resumir o que encontra

Não deve conseguir

- Mudar um projeto inteiro com segurança
- Planear tarefas longas de muitos passos sem se perder
- Saber APIs recentes ou bibliotecas pouco comuns
- Acertar factos com fiabilidade: vai inventar
- Competir com assistentes comerciais grandes

9 · MARCOS

Doze marcos, quatro fases

Como no 0.1: um marco de cada vez, cada um com testes e resultados medidos no PLAN. Marcos V1–V12, para não se confundirem com os K1–K10 do Vulpes Flos 1.0.

#	MARCO	ONDE	CUSTO
Fase A · Preparar, no teu PC (depois do Vulpes Flos 1.0)			
V1	**Plano e decisões** Este documento; responder a D1–D8	Desktop	0
V2	**Tokenizer v2** Amostra mista, 49 152 ids, tokens especiais, metas medidas	Desktop (CPU)	0
V3	**Dados v2** Fontes novas, filtros de código, segredos, FIM, descontaminação, mistura	Desktop + pod CPU	10–30 USD
V4	**Modelo e treino v2** Presets mini/lusitanus, θ, 4k/8k; DDP e WSD já vêm do Vulpes Flos 1.0; testes em CPU como sempre	Desktop	0
Fase B · Ensaio			
V5	**Benchmark em H100** Mede tokens/s reais em 1 e 8 GPUs e refaz as contas de custo	Pod, ~1 h	~10 USD
V6	**Piloto: Lusitanus Mini** 321M, 10B tokens, toda a cadeia até ao chat. Se falhar, para aqui	1× H100, ~1 dia	50–100 USD
Fase C · O modelo			
V7	**Lusitanus Base** 1,07B, 50B tokens; extensão a 100B decidida a meio (WSD)	8× H100, 2–4 dias	0,7–2,1 mil USD
V8	**Contexto 8k** ~2B tokens com documentos longos	H100, ~16 h	30–50 USD
V9	**Avaliação v2** Harness de execução, HumanEval, MBPP, teste em PT	Desktop	0
Fase D · O assistente			
V10	**Lusitanus Instruct (SFT)** Conversas EN/PT, commits, trajetórias de agente escritas por ti	H100, horas	10–30 USD
V11	**RL com testes** MBPP e APPS, 8 soluções por problema, sandbox	H100, ~1 dia	50–150 USD
V12	**Agente Vulpes e release** Ferramentas, confirmações, memória; pesos fp16/int8 para o PC	Desktop	0

Com o TRC (D8), V5–V11 correm em TPUs sem custo. O agente (V12) pode ser escrito a qualquer momento e testado com o Vulpes Flos 1.0.

10 · RISCOS

O que pode correr mal

O 0.1 correu bem porque cada risco tinha uma resposta antes de começar. O mesmo aqui.

RISCO	RESPOSTA
Não há orçamento nem TPUs	O Lusitanus espera. O Vulpes Flos 1.0 continua a ser o modelo do projeto, e o agente (V12) pode avançar com ele.
O custo sai acima do previsto	O V5 mede a velocidade real antes de qualquer gasto grande. `--stop-after` corta no orçamento. Com WSD, parar aos 50B dá um modelo acabado, não um treino a meio.
Perder um pod a meio	Checkpoints a cada ~30 min, copiados para o PC pelo `sync.py` com SHA-256. Retoma bit-a-bit noutra máquina, como no 0.1.
Picos na perda a 1B	LR mais baixa que no 0.1, aquecimento longo, clip 1,0; o piloto mostra se é preciso mais. Se acontecer, retoma-se do checkpoint anterior e salta-se o lote.
8 GPUs não escalam bem	O V5 mede 1 contra 8. Se a eficiência for baixa, usa-se 1–2 GPUs mais tempo, pelo mesmo preço.
O modelo aprende segredos ou dados pessoais	Filtro de chaves e e-mails antes de tokenizar; amostras revistas à mão no V3.
Resultados inflacionados	Descontaminação por blocos de 13 palavras contra todos os testes; a parte de teste de MBPP e HumanEval nunca toca no treino.
O RL não arranca	Se o modelo acerta 0%, não há nada para reforçar. Começa-se pelos problemas mais fáceis do MBPP, depois do SFT, e só se avança quando ~10% passam.
O agente estraga ficheiros	Nada é escrito sem diferença mostrada e um “sim”. Sem rede e sem comandos arbitrários na v1. Pasta de trabalho fixa.
Licenças de código	Só licenças permissivas, com a lista de origens guardada; os pedidos de remoção do The Stack são respeitados.
Lento demais no GTX 1650 Super	int8, contexto menor por defeito, ou o agente noutra máquina. Mede-se no V12.

11 · DECISÕES EM ABERTO

O que é teu decidir

Cada decisão tem uma recomendação, mas nenhuma está tomada. As respostas entram na tabela de decisões do `PLAN`, como no 0.1.

D1 **A regra "sem dados gerados por LLMs" aplica-se às respostas do próprio Vulpes no RL?**

Recomendado Sim à regra para outros modelos; permitir as respostas do Lusitanus que passam testes. Sem isto não há fase 2 de treino.

D2 **Orçamento: 50B tokens (~1 000–1 600 USD) ou 100B (~1 850–2 900 USD)?**

Recomendado Aprovar só o piloto (~100 USD) agora. Começar o Base com 50B e decidir a extensão a meio, com a curva de perda à frente.

D3 **Uma H100 durante 2–4 semanas, ou oito durante 2–4 dias?**

Recomendado Oito, se o V5 mostrar boa eficiência: o mesmo preço, muito menos tempo exposto a falhas, e o PC livre mais cedo.

D4 **Português europeu ou do Brasil?**

Recomendado No pré-treino, aceitar a mistura do FineWeb-2 (há muito mais texto do Brasil). Identidade, trajetórias e respostas em português europeu, escritas por ti.

D5 **Dados share-alike (Dolly, StackExchange) entram?**

Como no 0.1 A mesma pergunta da decisão 31. O StackExchange ajudaria bastante em código; a licença é CC BY-SA, como a Wikipedia recusada na decisão 4.

D6 **Que linguagens de programação, e com que peso?**

Recomendado Python ~40% do código; JavaScript/TypeScript, shell, SQL, HTML/CSS e Markdown no resto. Linguagens que não usas ficam de fora: 1B não chega para tudo.

D7 **Os pesos do Vulpes Lusitanus 1.0 são publicados?**

Em aberto Como no 0.1 (GitHub Releases, Apache-2.0), ou privados. Influencia o cuidado com os termos do StarCoderData e com a escolha do teste em português. Com o TRC (D8), publicar passa a ser a contrapartida.

D8 **Onde treinar: GPUs alugadas ou TPU Research Cloud?**

Recomendado Candidatar ao TRC já, porque não custa nada e a resposta demora. Se for aceite, portar o treino para torch_xla (mais perto do código atual) ou JAX; se não, ficam as GPUs alugadas de D2–D3.

Próximo passo. O Lusitanus vem depois do Vulpes Flos 1.0 (plano à parte), que corre todo no Kaggle grátis. Até lá, só duas coisas: responder a D1–D8 quando quiseres e, se fizer sentido, a candidatura ao TRC. Com as respostas, este plano passa para `docs/` como o `PLAN.md` do Lusitanus e começa o V2 (tokenizer v2), que corre todo no teu PC e não custa nada.